

Лабораторная работа №3

Студент: Мансуров Роман

Группа: 6312-100503D

1. Цель работы

Изучение алгоритма умножения квадратных матриц и методов его распараллеливания с использованием технологии MPI. Получение практических навыков разработки параллельных программ, распределения данных между процессами, сбора результатов и анализа производительности (ускорения и эффективности) на многопроцессорных вычислительных системах.

2. Описание работы алгоритма

В данной работе реализован классический алгоритм умножения двух квадратных матриц с применением технологии MPI. Для матриц A и B размера $n \times n$ результирующая матрица $C = A * B$ вычисляется по стандартной формуле. Параллелизм достигается за счет распределения строк матрицы A между MPI-процессами.

Программа на C++:

1. Инициализация MPI-среды (MPI_Init).
2. Определение ранга процесса и общего количества процессов.
3. Процесс с рангом 0 (главный) считывает матрицы A и B из файлов data/matrix_a.txt и data/matrix_b.txt. Проверяется, что размер матриц n делится на количество процессов нацело.
4. Главный процесс рассылает размерность n всем процессам (MPI_Bcast).
5. Главный процесс рассылает всю матрицу B всем процессам (MPI_Bcast), так как она необходима каждому процессу для вычислений.
6. Главный процесс распределяет (разбивает) строки матрицы A между всеми процессами с помощью MPI_Scatter. Каждый процесс получает локальный фрагмент local_A размером $\text{rows_per_process} * n$.
7. Каждый процесс выполняет умножение своего фрагмента строк local_A на общую матрицу B, записывая результат в локальный фрагмент local_C.

8. Все процессы отправляют свои вычисленные фрагменты `local_C` главному процессу с помощью `MPI_Gather`. Главный процесс собирает их в результирующую матрицу `C`.
9. Замер времени (`MPI_Wtime`) производится на всех процессах, но итоговое время выводится только главным процессом, охватывая этапы распределения данных, вычислений и сбора результатов.
10. Главный процесс сохраняет результирующую матрицу в файл `data/result_cpp.txt` и выводит время выполнения.

Python-скрипт автоматизирует эксперимент:

1. Генерирует случайные матрицы для размеров 208, 400, 800, 1200, 1600, 2000.
2. Для каждого размера запускает MPI-программу сериями по 5 раз с разным количеством процессов (1, 2, 4, 8, 16). Для каждого размера проверяется условие делимости n на число процессов.
3. Собирает полученные времена выполнения, вычисляет среднее значение.
4. Строит и сохраняет график зависимости времени выполнения от размера матрицы для разного количества процессов.

3. Характеристики вычислительной системы

Процессор:

- Модель: AMD Ryzen 7 2700 Eight-Core Processor
- Количество ядер: 8
- Тактовая частота: 3.20 GHz

Оперативная память:

- Объем: 24 Гб
- Тип: DDR4

Операционная система:

- Название: Windows 11
- Разрядность: 64bit

4. Результаты экспериментов

Размер 208x208 Процессов: 1 0.0550 сек 0.0590 сек 0.0560 сек 0.0559 сек 0.0556 сек Процессов: 2 0.0287 сек 0.0284 сек 0.0281 сек 0.0282 сек 0.0285 сек Процессов: 4 0.0158 сек 0.0169 сек 0.0151 сек 0.0150 сек 0.0150 сек Процессов: 8 0.0138 сек 0.0143 сек 0.0144 сек 0.0143 сек 0.0087 сек Процессов: 16 0.0088 сек 0.0084 сек 0.0086 сек 0.0095 сек 0.0085 сек	Размер 400x400 Процессов: 1 0.3868 сек 0.3875 сек 0.3867 сек 0.3840 сек 0.3855 сек Процессов: 2 0.1961 сек 0.2188 сек 0.1957 сек 0.2011 сек 0.1964 сек Процессов: 4 0.1001 сек 0.0995 сек 0.1038 сек 0.1107 сек 0.1000 сек Процессов: 8 0.0829 сек 0.0583 сек 0.0840 сек 0.0833 сек 0.0780 сек Процессов: 16 0.0603 сек 0.0559 сек 0.0636 сек 0.0641 сек 0.0539 сек	Размер 800x800 Процессов: 1 3.1417 сек 3.1619 сек 3.0820 сек 3.0869 сек 3.0882 сек Процессов: 2 1.5745 сек 1.5737 сек 1.5590 сек 1.5672 сек 1.5946 сек Процессов: 4 0.8151 сек 0.8365 сек 0.8190 сек 0.8253 сек 0.8121 сек Процессов: 8 0.5311 сек 0.5214 сек 0.5506 сек 0.5204 сек 0.5038 сек Процессов: 16 0.4348 сек 0.4357 сек 0.4354 сек 0.4326 сек 0.4253 сек	Размер 1200x1200 Процессов: 1 10.8577 сек 10.8341 сек 10.8287 сек 10.7863 сек 10.8435 сек Процессов: 2 5.4848 сек 5.5055 сек 5.4664 сек 5.4622 сек 5.4705 сек Процессов: 4 2.9352 сек 2.8594 сек 2.8329 сек 2.8176 сек 2.7943 сек Процессов: 8 1.7336 сек 2.1739 сек 1.7390 сек 1.7280 сек 1.9432 сек Процессов: 16 1.4178 сек 1.4560 сек 1.4192 сек 1.4232 сек 1.4485 сек
Размер 1600x1600 Процессов: 1 25.9181 сек 25.9392 сек 26.0869 сек 26.0932 сек 25.8971 сек Процессов: 2 13.0632 сек 13.2185 сек 13.2333 сек 13.1709 сек 13.1652 сек Процессов: 4 6.6659 сек 6.7104 сек 6.6990 сек 6.7717 сек 6.7200 сек Процессов: 8 3.9548 сек 4.0271 сек 4.0059 сек 4.0764 сек 4.0329 сек Процессов: 16 3.3915 сек 3.3624 сек 3.9341 сек 4.4164 сек	Размер 2000x2000 Процессов: 1 53.4635 сек 51.2496 сек 51.0618 сек 51.0670 сек 51.1356 сек Процессов: 2 25.8917 сек 26.2268 сек 25.8958 сек 25.9565 сек 25.8954 сек Процессов: 4 13.2387 сек 13.2409 сек 13.2311 сек 13.3326 сек 13.2198 сек Процессов: 8 7.8059 сек 7.7846 сек 7.6057 сек 7.6982 сек 8.2746 сек Процессов: 16 6.5626 сек 6.5685 сек 6.5948 сек 6.6764 сек		

3.7826 сек	6.6032 сек
------------	------------

Размер 208x208 Процессов: 1 → 0.0563 сек (±0.0016) Процессов: 2 → 0.0284 сек (±0.0002) Процессов: 4 → 0.0156 сек (±0.0008) Процессов: 8 → 0.0131 сек (±0.0021) * Процессов: 16 → 0.0088 сек (±0.0004)	Размер 400x400 Процессов: 1 → 0.3861 сек (±0.0013) Процессов: 2 → 0.2016 сек (±0.0088) Процессов: 4 → 0.1028 сек (±0.0043) Процессов: 8 → 0.0773 сек (±0.0094) Процессов: 16 → 0.0596 сек (±0.0039)	Размер 800x800 Процессов: 1 → 3.1121 сек (±0.0292) Процессов: 2 → 1.5738 сек (±0.0115) Процессов: 4 → 0.8216 сек (±0.0085) Процессов: 8 → 0.5255 сек (±0.0155) Процессов: 16 → 0.4328 сек (±0.0039)
Размер 1200x1200 Процессов: 1 → 10.8301 сек (±0.0229) Процессов: 2 → 5.4779 сек (±0.0150) Процессов: 4 → 2.8479 сек (±0.0489) Процессов: 8 → 1.8635 сек (±0.1577) Процессов: 16 → 1.4329 сек (±0.0156)	Размер 1600x1600 Процессов: 1 → 25.9669 сек (±0.0800) Процессов: 2 → 13.1702 сек (±0.0582) Процессов: 4 → 6.7134 сек (±0.0344) Процессов: 8 → 4.0194 сек (±0.0409) Процессов: 16 → 3.7774 сек (±0.3759) *	Размер 2000x2000 Процессов: 1 → 51.5955 сек (±0.9367) * Процессов: 2 → 25.9732 сек (±0.1176) Процессов: 4 → 13.2526 сек (±0.0411) Процессов: 8 → 7.8338 сек (±0.2222) Процессов: 16 → 6.6011 сек (±0.0400)

* - небольшой разброс значений, связанный с фоновой нагрузкой ОС или вариативностью MPI-коммуникаций.

На больших размерах матриц (800, 1200, 1600, 2000) наблюдается хорошее ускорение при увеличении числа процессов от 1 до 8 (что соответствует числу физических ядер процессора). Например, для размера 2000 время выполнения сокращается с ~51.6 секунды (1 процесс) до ~7.8 секунды (8 процессов). Ускорение составляет около 6.6 раз.

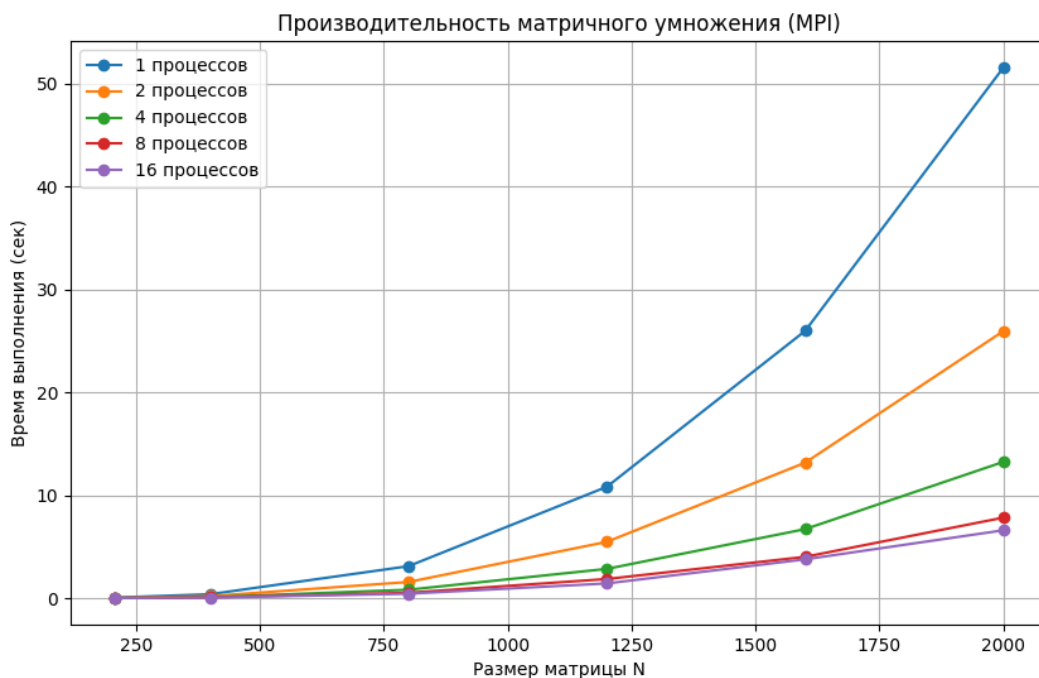
При переходе с 8 на 16 процессов прирост производительности замедляется. Для размера 2000 ускорение при 16 процессах относительно 1 процесса составляет ~7.8 раз, что лишь незначительно (на ~15%) лучше, чем для 8 процессов. Это объясняется накладными расходами на межпроцессное взаимодействие (коммуникации MPI) и тем, что 16 процессов делят ресурсы 8 физических ядер. Для размера 1600 наблюдался даже более высокий разброс результатов для 16 процессов, что указывает на возросшую конкуренцию за ресурсы.

На небольшой матрице (208x208) выигрыш от увеличения числа процессов ограничен, так как время непосредственных вычислений становится сравнимо с временем, затрачиваемым на коммуникационные операции (MPI_Bcast, MPI_Scatter, MPI_Gather).

Однако даже в этом случае наблюдается значительное ускорение (в ~ 6.4 раза при 16 процессах), что говорит об эффективной реализации.

Доверительные интервалы для большинства замеров узкие (доли процента от среднего значения), что свидетельствует о высокой стабильности выполнения параллельного алгоритма на MPI и низком влиянии фоновых процессов.

5. График



6. Вывод

В ходе лабораторной работы была разработана и исследована параллельная версия алгоритма умножения квадратных матриц с использованием технологии MPI. Эксперименты подтвердили, что распределение вычислений между несколькими процессами позволяет значительно сократить время решения вычислительно трудоемких задач на многопроцессорных системах.

Установлено, что для использованного процессора AMD Ryzen 7 2700 (8 физических ядер) оптимальное ускорение достигается при использовании 8 процессов. Дальнейшее увеличение количества процессов до 16 не обеспечивает пропорционального выигрыша из-за накладных расходов на коммуникацию и конкуренции за ресурсы, а в некоторых случаях может даже ухудшить результат из-за возросшей вариативности. Разработанный комплекс из C++ MPI-программы и Python-скрипта lab.py позволил полностью автоматизировать процесс сбора данных, обеспечил проверку корректности работы и наглядно продемонстрировал характеристики масштабируемости параллельного алгоритма.

7. Исходный код

Исходный код представлен в репозитории на GitHub.